Fetch Data with AWS Lambda

Mohammed Dawoud Mota

Introducing Today's Project!

In this project, I set out to build a serverless workflow that retrieves data from a DynamoDB table using an AWS Lambda function. My objective was to gain practical experience with the data tier of a three tier architecture and to strengthen my understanding of how cloud native services operate together in a scalable environment. By working with AWS Lambda and Amazon DynamoDB, I aimed to develop the ability to design backend solutions that run efficiently without the need to manage any underlying servers. This project allowed me to apply core serverless concepts while building a reliable data retrieval process that reflects real world application behavior.

Tools and concepts

I learned several key services and concepts in this project because I had to connect a Lambda function to DynamoDB, troubleshoot permissions, and understand how AWS handles identity and access. I worked directly with AWS Lambda to run my code without managing servers, Amazon DynamoDB as my NoSQL database, and the AWS SDK to make programmatic calls from my function. I also gained a solid understanding of IAM roles and least-privilege access, especially how execution roles determine what a Lambda function is allowed to do. Altogether, this helped me strengthen my cloud fundamentals and learn how serverless components interact securely and efficiently.

Project reflection

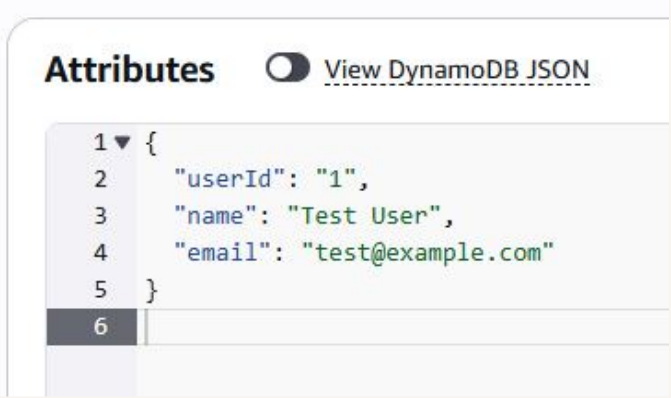
It took me only a short amount of time to complete this project because once I understood how AWS Lambda, IAM roles, and DynamoDB fit together, the rest was just testing, fixing the access-denied issue, and confirming that my function could successfully read data using the AWS SDK, so the whole workflow came together pretty quickly.

I chose to do this project today because I wanted to continue working on the three-tier architecture.

Project Setup

To set up the project, I created a DynamoDB table named UserData that will serve as the primary data store for the application. The table uses userId as its partition key, which ensures that each record is uniquely identifiable and can be retrieved efficiently.

I added a sample user record to the UserData table so I could later validate that the Lambda function retrieves data correctly. The item included a userId of 1, along with a name and email address, which gave the table meaningful data for testing.



AWS Lambda

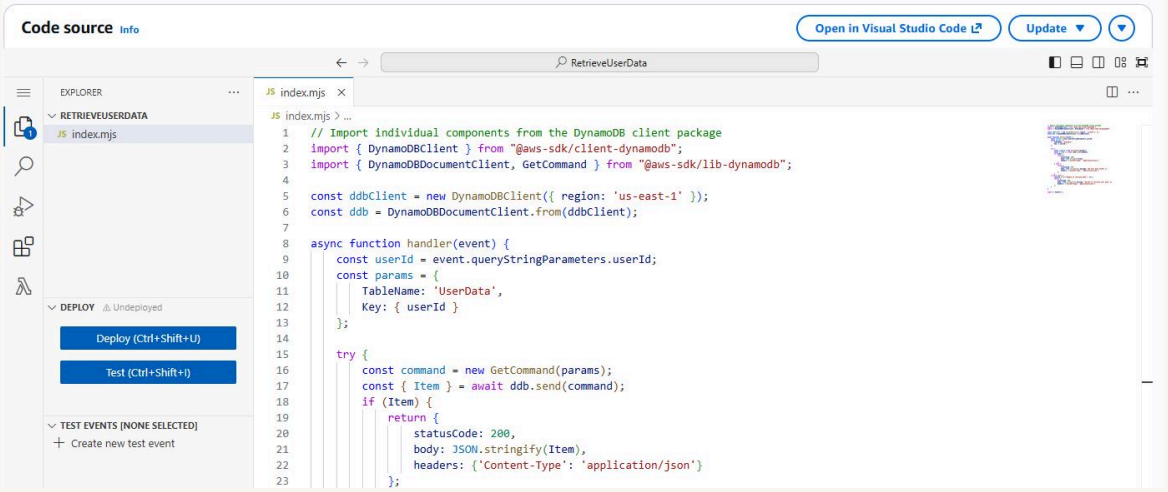
AWS Lambda is a serverless compute service that allows me to run code without provisioning or managing any underlying servers. It automatically handles scaling, availability, and execution so the focus stays entirely on the application logic rather than the infrastructure. In this project, I am using AWS Lambda to create a lightweight backend function that retrieves user data from DynamoDB on demand. This makes the solution efficient, cost-effective, and capable of supporting workloads that scale from a single request to thousands per second

AWS Lambda Function

My function's execution role defines the specific AWS resources and actions that the Lambda function is permitted to use. It acts as a security boundary that ensures the function operates with the least privilege necessary. By default, the role allows the function to write logs to CloudWatch, which is essential for monitoring and troubleshooting. As the project progressed, I expanded this role to include the permissions required to read data from Amazon DynamoDB so the function could securely access the UserData table without granting any unnecessary or overly broad access.

My code block is responsible for retrieving a specific user's data from the DynamoDB table based on the userId provided in the event input. It begins by creating a DynamoDB client through the AWS SDK and then constructs a GetCommand request targeting the UserData table. When the function runs, it extracts the userId from the incoming event, sends the request to DynamoDB, and returns the matching item if it exists. The code also includes structured error handling to ensure that any issues during the database call are logged clearly, allowing the function to fail safely while providing meaningful diagnostic information. This logic forms the core of how the Lambda function interacts with Amazon DynamoDB to support data retrieval within a serverless architecture.

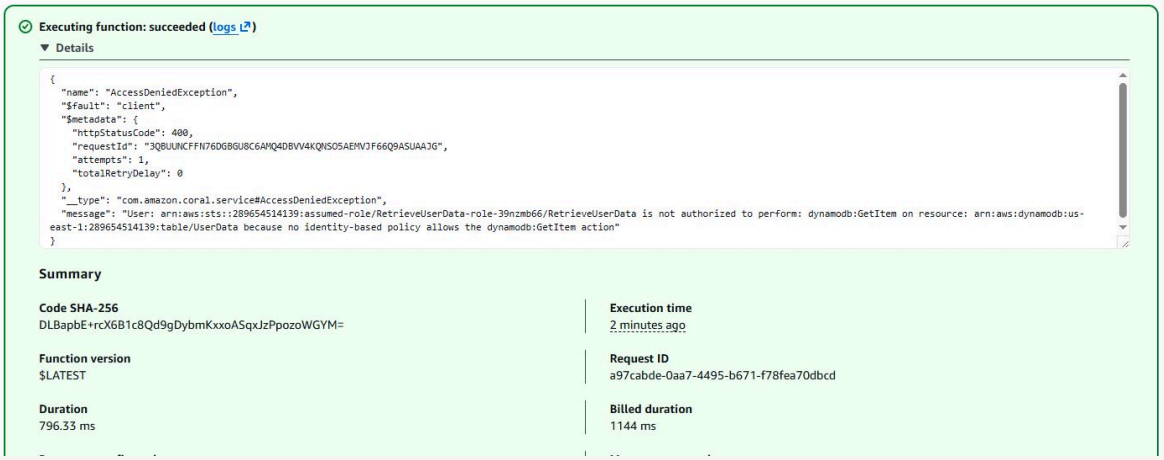
The AWS SDK is a collection of software tools and libraries that allows applications to interact directly with AWS services through prebuilt, reliable APIs. Instead of writing low-level code to communicate with AWS, the SDK provides structured methods that make it easy to perform tasks such as reading from DynamoDB, invoking Lambda functions, or interacting with other cloud resources.


Code source Info
Open in Visual Studio Code
Update
RetrieveUserData
EXPLORER
RETRIEVEUSERDATA
index.mjs
DEPLOY
Undeployed
Deploy (Ctrl+Shift+U)
Test (Ctrl+Shift+I)
TEST EVENTS [NONE SELECTED]
Create new test event
// Import individual components from the DynamoDB client package
import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
const ddbClient = new DynamoDBClient({ region: 'us-east-1' });
const ddb = DynamoDBDocumentClient.from(ddbClient);
async function handler(event) {
const userId = event.queryStringParameters.userId;
const params = {
TableName: 'UserData',
Key: { userId }
};
try {
const command = new GetCommand(params);
const { Item } = await ddb.send(command);
if (Item) {
return {
statusCode: 200,
body: JSON.stringify(Item),
headers: {'Content-Type': 'application/json'}
};

Function Testing

I tested my Lambda function by invoking it with a custom test event in the Lambda console, using a JSON payload that included the same userId I inserted into the DynamoDB table; this allowed me to simulate a real request, confirm that the function could read the input, call Amazon DynamoDB through the AWS SDK, and return the correct user record while also verifying that execution details were logged properly in CloudWatch.

I ran into an error because I hadn't given my Lambda function the permissions it needed to read from my DynamoDB table, so when the function tried to access the data, AWS blocked it with an AccessDeniedException. My execution role only had the basic CloudWatch logging policy at that point, so the moment my code used the AWS SDK to call DynamoDB, the request failed. Once I updated the role to include the correct DynamoDB read permissions, the function was finally allowed to access the table and the error disappeared.

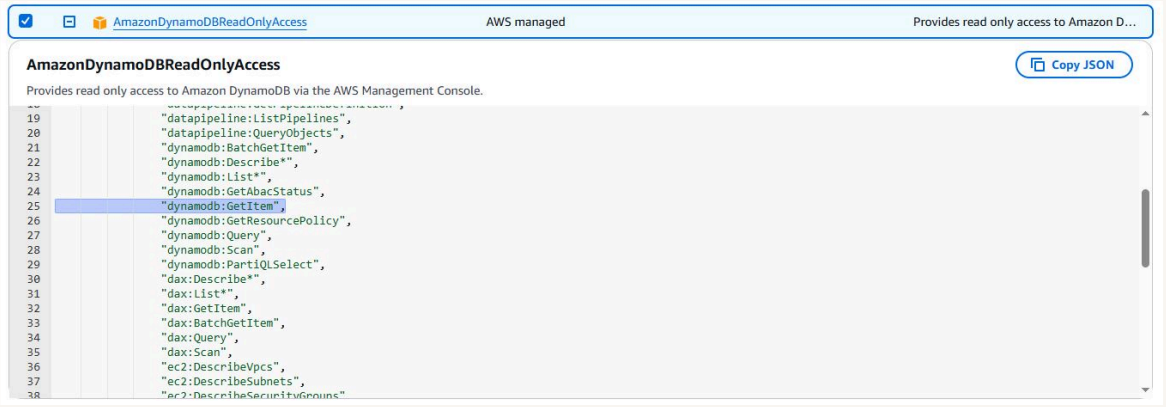

Executing function: succeeded (logs)
Details
{
"name": "AccessDeniedException",
"$fault": "client",
"$metadata": {
"httpStatusCode": 400,
"requestId": "3QBUUNCFFN76DGBGU8C6AMQ4DBVV4KQNSO5AEMVJF66Q9ASUAAJG",
"attempts": 1,
"totalRetryDelay": 0
},
"__type": "com.amazon.coral.service#AccessDeniedException",
"message": "User: arn:aws:sts::289654514139:assumed-role/RetrieveUserData-role-39nzmb66/RetrieveUserData is not authorized to perform: dynamodb:GetItem on resource: arn:aws:dynamodb:us-east-1:289654514139:table/UserData because no identity-based policy allows the dynamodb:GetItem action"
}
Summary
Code SHA-256
DLBapbE+rcX6B1c8Qd9gDybmKxxoASqxJzPpozoWGYM=
Execution time
2 minutes ago
Function version
$LATEST
Request ID
a97cabde-0aa7-4495-b671-f78fea70dbcd
Duration
796.33 ms
Billed duration
1144 ms

Function Permissions

I’m resolving the access-denied error by updating my Lambda function’s execution role to include the AmazonDynamoDBReadOnlyAccess policy, which gives my function permission to read from my DynamoDB table; once I attach that policy, my Lambda can finally call DynamoDB through the AWS SDK without AWS blocking the request.

I wouldn’t select those two policies because they give way more access than I actually need, and I’m trying to keep my Lambda execution role as tight and least-privilege as possible. The broad admin-level DynamoDB policies would let my function write, delete, or even manage tables, which is completely unnecessary for what I’m doing. All I need is the ability to read a single item from Amazon DynamoDB using the AWS SDK, so choosing anything more powerful would just increase the security risk without providing any benefit.

I picked AmazonDynamoDBReadOnlyAccess because all I need right now is the ability to read data from my DynamoDB table, nothing more. Choosing this policy keeps my Lambda execution role aligned with least-privilege best practices, avoids giving unnecessary write or admin permissions, and still lets my function call Amazon DynamoDB through the AWS SDK without running into access-denied errors.


AmazonDynamoDBReadOnlyAccess
AWS managed
Provides read only access to Amazon D...
AmazonDynamoDBReadOnlyAccess
Copy JSON
Provides read only access to Amazon DynamoDB via the AWS Management Console.
"datapipeline:ListPipelines",
"datapipeline:QueryObjects",
"dynamodb:BatchGetItem",
"dynamodb:Describe*",
"dynamodb:List*",
"dynamodb:GetAbacStatus",
"dynamodb:GetItem",
"dynamodb:GetResourcePolicy",
"dynamodb:Query",
"dynamodb:Scan",
"dynamodb:PartiQLSelect",
"dax:Describe*",
"dax:List*",
"dax:GetItem",
"dax:BatchGetItem",
"dax:Query",
"dax:Scan",
"ec2:DescribeVpcs",
"ec2:DescribeSubnets",

Final Testing and Reflection

I saw a successful result because I finally gave my Lambda function the correct permissions, so when it ran this time, it was actually allowed to read from my DynamoDB table. With the AmazonDynamoDBReadOnlyAccess policy attached, my function could call Amazon DynamoDB through the AWS SDK without getting blocked, which let it retrieve the item and return the expected output.

I can use what I've learned in a lot of useful ways because now I understand how to connect a Lambda function to DynamoDB, how to troubleshoot permission issues, and how to apply least privilege when working with the AWS SDK, so I can reuse this same pattern whenever I build serverless APIs, automate backend tasks, or create event driven workflows that rely on AWS services working together.

The place to learn & showcase your skills

Check out nextwork.org for more projects